

Feature Engineering Notebook for my CTA strategy

In [1]: *# Import packages*

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [2]: *# Read data*

```
folder_path = r"C:\Users\pcarg\OneDrive\Υπολογιστής\MFE UCLA\Projects\Commodities\D

open_prices = pd.read_parquet(f"{folder_path}\\open.parquet")
high_prices = pd.read_parquet(f"{folder_path}\\high.parquet")
low_prices = pd.read_parquet(f"{folder_path}\\low.parquet")
close_prices = pd.read_parquet(f"{folder_path}\\close.parquet")
volume = pd.read_parquet(f"{folder_path}\\volume.parquet")
volatility = pd.read_parquet(f"{folder_path}\\yang_zhang_features.parquet")
```

In [5]: *# Remove known bad Rice print Yahoo data error around 2024-06-17*

```
bad_date = "2024-06-17"

for df in [
    open_prices,
    high_prices,
    low_prices,
    close_prices
]:
    df.loc[bad_date, "Rice"] = np.nan
```

In [7]: *# Sanity check*

```
print(open_prices.shape)
print(high_prices.shape)
print(low_prices.shape)
print(close_prices.shape)
print(volume.shape)

print(open_prices.columns.equals(close_prices.columns))
print(open_prices.index.equals(close_prices.index))
```

(6475, 38)

(6475, 38)

(6475, 38)

(6475, 38)

(6475, 38)

True

True

```
In [9]: # Extract volatility horizons

yz_vol_21 = volatility["yz_vol_21"]
yz_vol_63 = volatility["yz_vol_63"]
yz_vol_126 = volatility["yz_vol_126"]
yz_vol_252 = volatility["yz_vol_252"]
```

Let's think about what we want to do: A preliminary feature set is the following

- Momentum: 21, 63, 126, 252
- Moving Average Features: 50, 100, 200
- Trend t-stat: Use slope over 63, 126 and 252 days
- Donchian breakouts: 20, 60, 120 days
- Volatility: 21, 63, 252, try Parkinson, Yang-Zhang
- Any RV crap I'll think about

Time-Series Momentum Alphas

As a first step, try to look at multiple momentum horizons with our without lookback periods. All features make it.

```
In [14]: # =====
# Raw Momentum Features
# =====

lookbacks = [21, 63, 126, 252]

momentum = {}

for lb in lookbacks:
    momentum[f"mom_{lb}"] = (
        close_prices / close_prices.shift(lb) - 1
    )

# =====
# Skip-Momentum Features
# Skip period scales from the
# classical 12-1 specification:
# skip = lookback * (21 / 252)
# =====

for lb in lookbacks:

    skip = max(1, round(lb * 21 / 252))

    momentum[f"mom_{lb}_skip"] = (
        close_prices.shift(skip)
        / close_prices.shift(lb)
```

```

        - 1
    )

# =====
# Individual DataFrames
# =====

mom_21 = momentum["mom_21"]
mom_63 = momentum["mom_63"]
mom_126 = momentum["mom_126"]
mom_252 = momentum["mom_252"]

mom_21_skip = momentum["mom_21_skip"]
mom_63_skip = momentum["mom_63_skip"]
mom_126_skip = momentum["mom_126_skip"]
mom_252_skip = momentum["mom_252_skip"]

# =====
# Combined Feature Store
# =====

features = pd.concat(momentum, axis=1)

```

Now we transition from features to signals. We will evaluate 3 distinct ways, each with their pros and cons:

- Binary: Only show direction (+1, 0, -1). Very robust, but throws away information
- Continuous: For time series momentum, we use cross-sectional z-scores
- Ranked: Robust, but loses valuable absolute information

```

In [17]: # Generate binary signals

binary_signals = {
    name: np.sign(feature)
    for name, feature in momentum.items()
}

```

```

In [19]: # Generate continuous signals, which are the vol normalized TSMOM from Ooi - Peders

continuous_signals = {
    "mom_21": mom_21 / yz_vol_21,
    "mom_63": mom_63 / yz_vol_63,
    "mom_126": mom_126 / yz_vol_126,
    "mom_252": mom_252 / yz_vol_252,

    "mom_21_skip": mom_21_skip / yz_vol_21,
    "mom_63_skip": mom_63_skip / yz_vol_63,
    "mom_126_skip": mom_126_skip / yz_vol_126,
    "mom_252_skip": mom_252_skip / yz_vol_252,
}

```

```
In [21]: # Generate ranked singals, which means convert features into percentile ranks: We a

rank_signals = {
    name: feature.rank(axis=1, pct=True) - 0.5
    for name, feature in momentum.items()
}
```

Now organize

```
In [24]: signals = {
    "binary": binary_signals,
    "continuous": continuous_signals,
    "rank": rank_signals
}
```

At this stage, the feature engineering process is complete and momentum features have been transformed into trading signals. The objective of the next stage is to determine whether these signals contain predictive information and whether they can generate profitable trading strategies. Importantly, this stage focuses on signal evaluation, not portfolio optimization. The goal is to understand the behavior of individual signals before combining them into a larger portfolio.

Research Questions

The initial backtests seek to answer the following questions:

1. Which momentum horizon performs best?
 - 21-day
 - 63-day
 - 126-day
 - 252-day
2. Does skipping recent returns improve performance?
3. Do continuous signals outperform binary signals?
4. How sensitive are results to rebalancing frequency?
5. Which signals deserve inclusion in a later multi-alpha portfolio?

Only after these questions are answered will more advanced portfolio construction techniques such as volatility targeting, risk budgeting, covariance estimation, and optimization be introduced.

```
In [29]: # =====
# Returns
# =====
```

```

returns = close_prices.pct_change(fill_method = None)

# =====
# Weekly Rebalancing
# =====

def weekly_positions(signal):

    weekly_signal = (
        signal
        .resample("W-FRI")
        .last()
    )

    positions = (
        weekly_signal
        .reindex(signal.index)
        .ffill()
    )

    return positions

# =====
# Monthly Rebalancing
# =====

def monthly_positions(signal):

    monthly_signal = (
        signal
        .resample("ME")
        .last()
    )

    positions = (
        monthly_signal
        .reindex(signal.index)
        .ffill()
    )

    return positions

# =====
# Backtest Function
# =====

def backtest_signal(signal, frequency="monthly"):

    # Rebalance schedule
    if frequency == "weekly":
        positions = weekly_positions(signal)

    elif frequency == "monthly":

```

```

        positions = monthly_positions(signal)

    else:
        raise ValueError(
            "frequency must be 'weekly' or 'monthly'"
        )

    # Avoid Look-ahead bias
    positions = positions.shift(1)

    # Gross exposure normalization
    # Sum(|weights|) = 1 every day
    gross_exposure = positions.abs().sum(axis=1)

    positions = positions.div(
        gross_exposure,
        axis=0
    )

    positions = positions.fillna(0)

    # Strategy returns
    strategy_returns = positions * returns

    portfolio_returns = (
        strategy_returns
        .sum(axis=1)
        .dropna()
    )

    return portfolio_returns

```

```

In [31]: # =====
# Binary Signals
# =====

binary_weekly = {}
binary_monthly = {}

for name, signal in signals["binary"].items():

    binary_weekly[name] = backtest_signal(
        signal,
        frequency="weekly"
    )

    binary_monthly[name] = backtest_signal(
        signal,
        frequency="monthly"
    )

```

```

In [32]: # =====
# Continuous Signals
# =====

```

```

continuous_weekly = {}
continuous_monthly = {}

for name, signal in signals["continuous"].items():

    continuous_weekly[name] = backtest_signal(
        signal,
        frequency="weekly"
    )

    continuous_monthly[name] = backtest_signal(
        signal,
        frequency="monthly"
    )

```

```

In [47]: rank_weekly = {}
rank_monthly = {}

for name, signal in signals["rank"].items():

    rank_weekly[name] = backtest_signal(
        signal,
        frequency="weekly"
    )

    rank_monthly[name] = backtest_signal(
        signal,
        frequency="monthly"
    )

```

```

In [49]: # =====
# Performance Statistics
# =====

def performance_stats(returns):

    ann_return = (
        (1 + returns).prod()
        ** (252 / len(returns))
        - 1
    )

    ann_vol = (
        returns.std()
        * np.sqrt(252)
    )

    sharpe = (
        ann_return / ann_vol
        if ann_vol > 0
        else np.nan
    )

    wealth = (
        1 + returns

```

```

).cumprod()

drawdown = (
    wealth / wealth.cummax()
    - 1
)

max_dd = drawdown.min()

return pd.Series({
    "Ann Return": ann_return,
    "Ann Vol": ann_vol,
    "Sharpe": sharpe,
    "Max DD": max_dd
})

```

```

In [51]: binary_monthly_summary = pd.DataFrame({
    name: performance_stats(ret)
    for name, ret in binary_monthly.items()
}).T

binary_monthly_summary.sort_values(
    "Sharpe",
    ascending=False
)

```

Out[51]:

	Ann Return	Ann Vol	Sharpe	Max DD
mom_252_skip	-0.001809	0.098692	-0.018335	-0.376817
mom_252	-0.003786	0.099885	-0.037906	-0.340434
mom_63_skip	-0.004719	0.097359	-0.048466	-0.357473
mom_126	-0.009890	0.096388	-0.102610	-0.323095
mom_126_skip	-0.010126	0.096800	-0.104609	-0.348789
mom_63	-0.013290	0.097109	-0.136853	-0.386347
mom_21	-0.018946	0.094216	-0.201091	-0.454886
mom_21_skip	-0.020429	0.094023	-0.217277	-0.462364

```

In [57]: plt.figure(figsize=(12, 6))

for name, ret in binary_monthly.items():

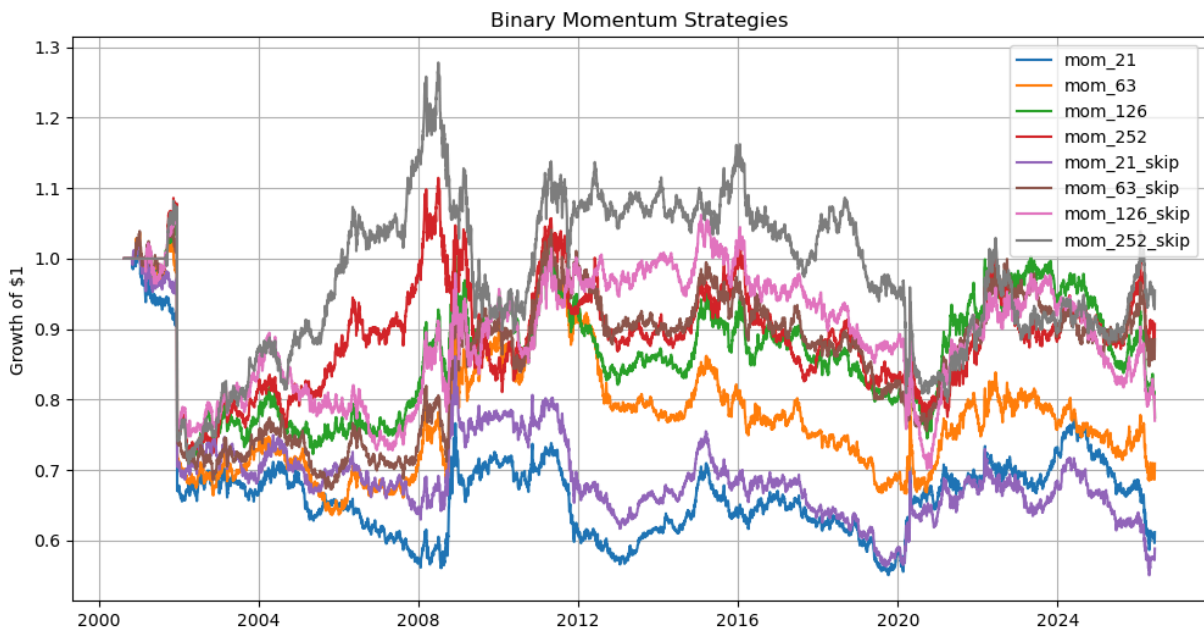
    equity_curve = (1 + ret).cumprod()

    plt.plot(
        equity_curve.index,
        equity_curve.values,
        label=name
    )

```



```
plt.title("Binary Momentum Strategies")
plt.ylabel("Growth of $1")
plt.legend()
plt.grid(True)
plt.show()
```



Takeaway Number 1: Binary momentum signals are a disaster in this universe. And generally, this kind of makes sense, because it does not quantify the signal. It throws a lot of information out. And this makes stuff not work.

```
In [54]: continuous_monthly_summary = pd.DataFrame({
    name: performance_stats(ret)
    for name, ret in continuous_monthly.items()
}).T

continuous_monthly_summary.sort_values(
    "Sharpe",
    ascending=False
)
```

Out[54]:

	Ann Return	Ann Vol	Sharpe	Max DD
mom_126_skip	0.013714	0.110350	0.124275	-0.340483
mom_252_skip	0.007916	0.103192	0.076713	-0.508912
mom_252	0.006118	0.107522	0.056900	-0.491368
mom_63_skip	0.003815	0.116793	0.032666	-0.364872
mom_126	-0.002152	0.117597	-0.018299	-0.381097
mom_21_skip	-0.005634	0.102791	-0.054812	-0.362482
mom_63	-0.007066	0.119827	-0.058965	-0.435899
mom_21	-0.009676	0.106274	-0.091045	-0.407731

Takeaway Number 2: Continuous signal seems to be doing better. What we can instantly see before anything else, is that trends that have long horizons seem to be performing better. Would it make sense to have momentum with a 21 day horizon that works? I don't think so.

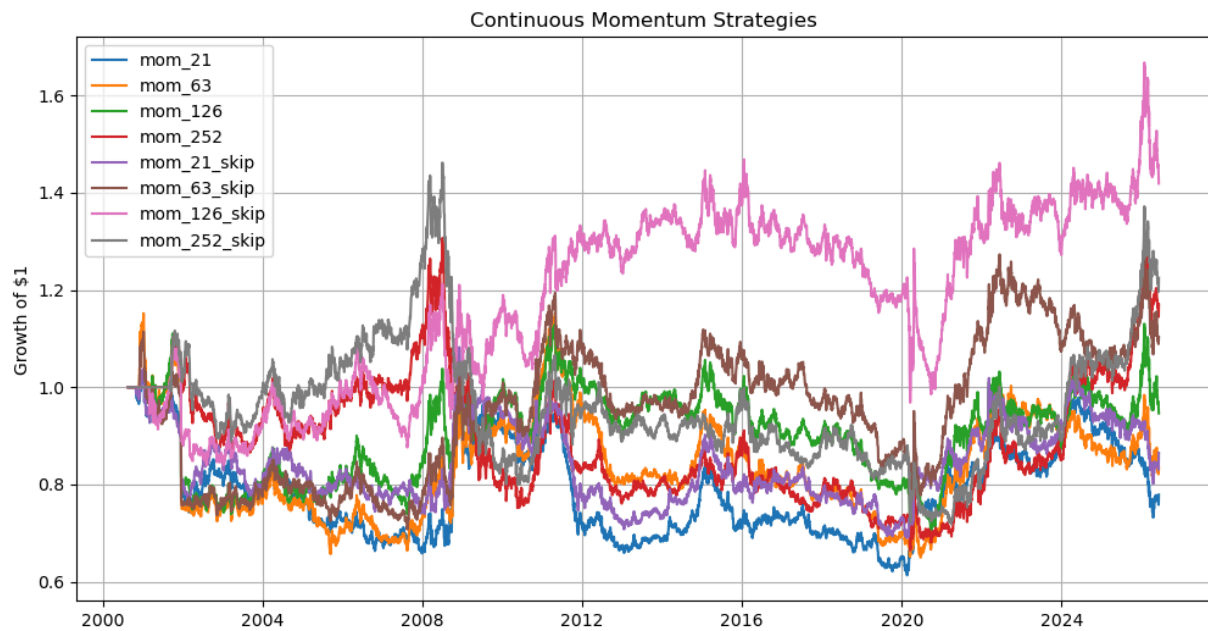
```
In [60]: plt.figure(figsize=(12, 6))

for name, ret in continuous_monthly.items():

    equity_curve = (1 + ret).cumprod()

    plt.plot(
        equity_curve.index,
        equity_curve.values,
        label=name
    )

plt.title("Continuous Momentum Strategies")
plt.ylabel("Growth of $1")
plt.legend()
plt.grid(True)
plt.show()
```



In []:

In []:

In []:

Now let's take a look at our best performing signal.

```
In [45]: best = continuous_monthly["mom_252_skip"]
```

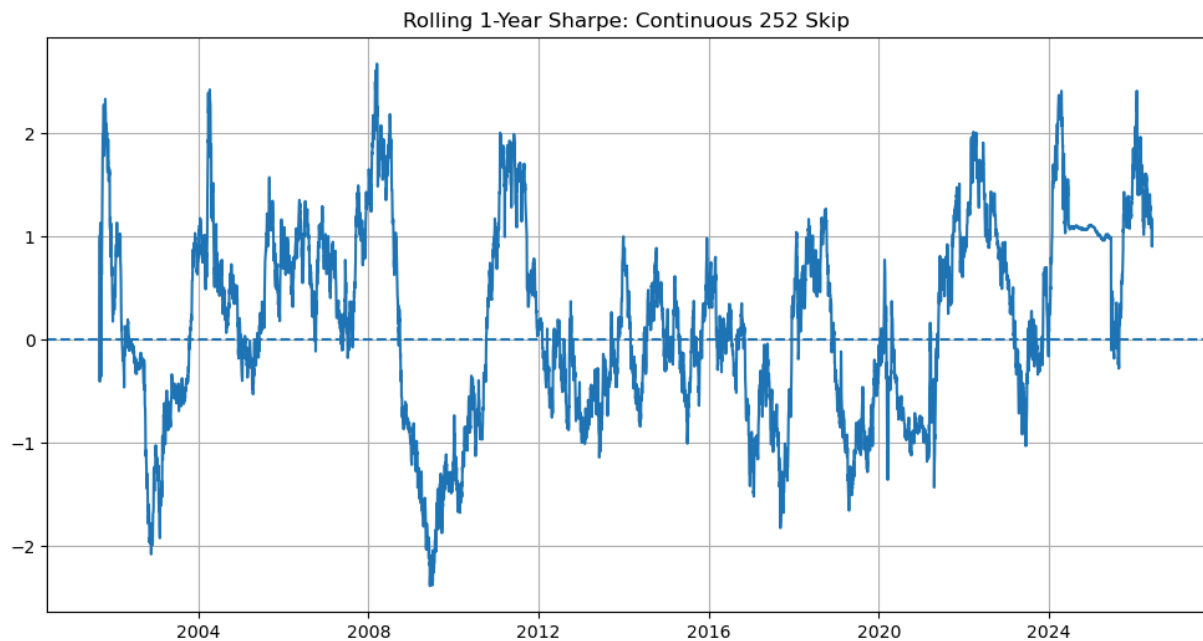
```
In [47]: rolling_sharpe = (
    best.rolling(252).mean()
    /
    best.rolling(252).std()
) * np.sqrt(252)

plt.figure(figsize=(12, 6))

plt.plot(
    rolling_sharpe.index,
    rolling_sharpe
)

plt.title("Rolling 1-Year Sharpe: Continuous 252 Skip")
plt.axhline(0, linestyle="--")
plt.grid(True)

plt.show()
```



Now let's look at the asset level performance of the strategy.

```
In [50]: signal = signals["continuous"]["mom_252_skip"]

positions = monthly_positions(signal)

positions = positions.shift(1)

positions = positions.div(
    positions.abs().sum(axis=1),
    axis=0
)

positions = positions.fillna(0)
```

```
In [52]: asset_pnl = (
    positions
    * returns
).sum()
```

```
In [54]: asset_pnl = asset_pnl.sort_values()
```

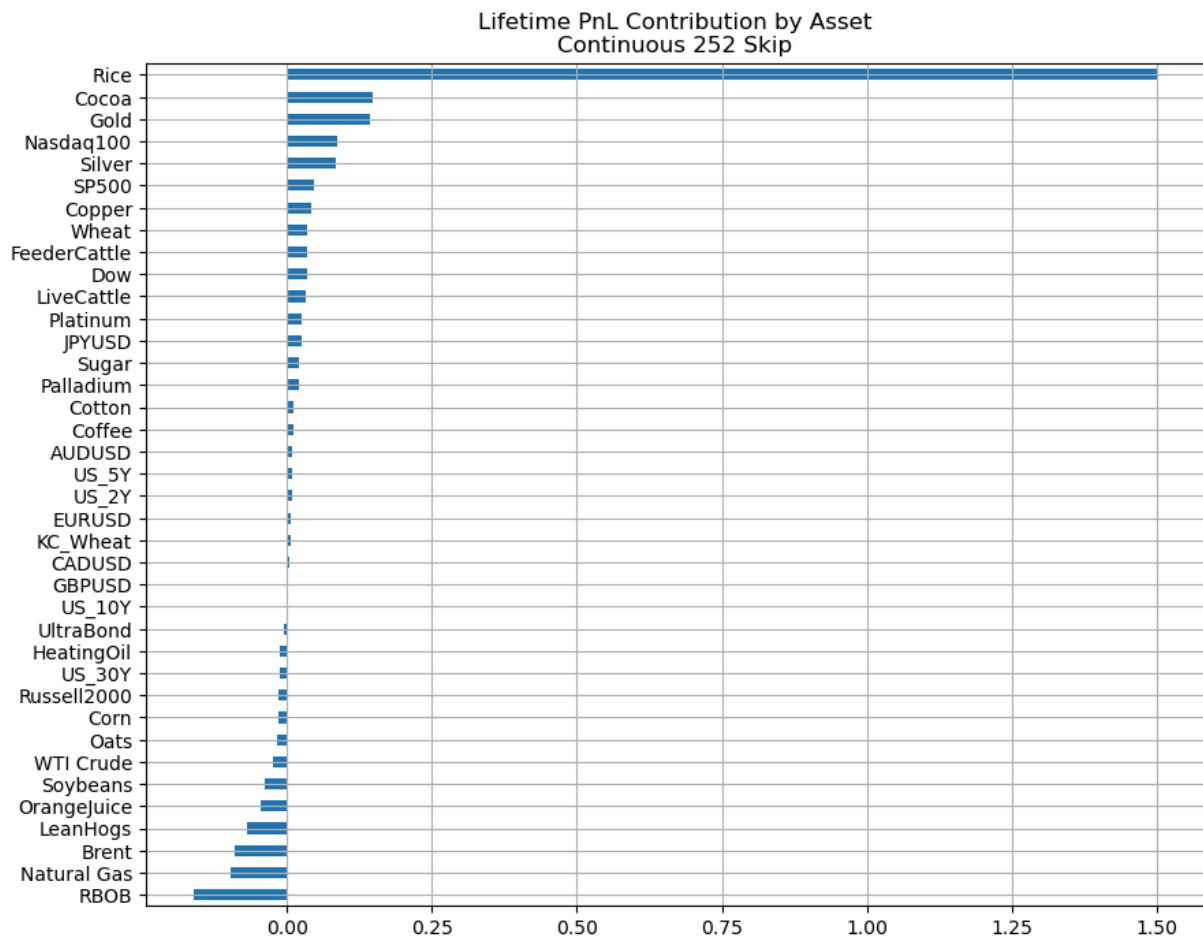
```
In [56]: plt.figure(figsize=(10, 8))

asset_pnl.plot(kind="barh")

plt.title(
    "Lifetime PnL Contribution by Asset\nContinuous 252 Skip"
)

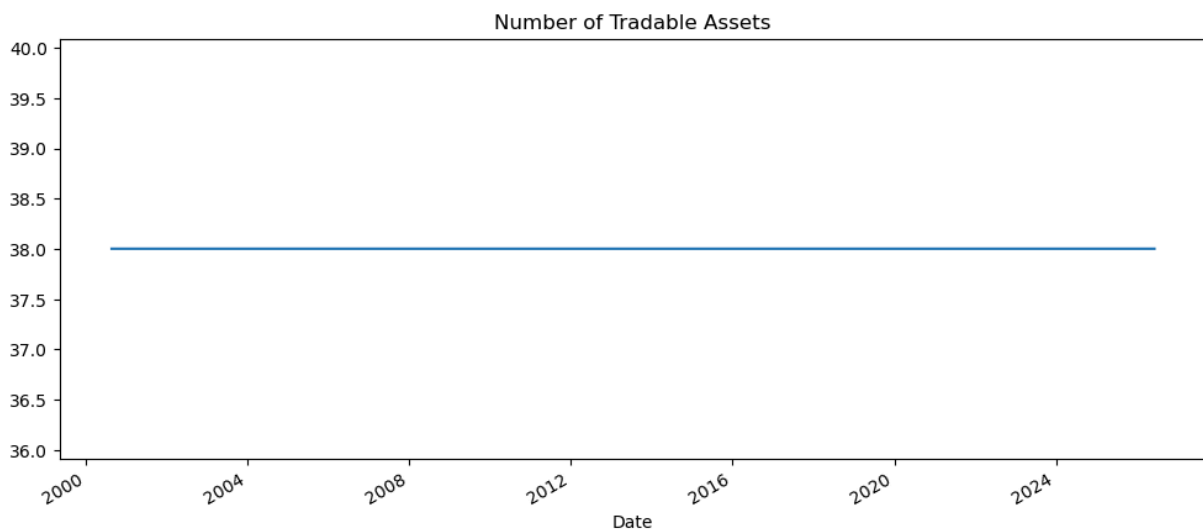
plt.grid(True)

plt.show()
```



```
In [58]: active_assets = (
    positions.notna()
    .sum(axis=1)
)

active_assets.plot(figsize=(12,5))
plt.title("Number of Tradable Assets")
plt.show()
```



Run backtest again from 2010.

```
In [61]: start_date = "2010-01-01"

signal = (
    signals["continuous"]["mom_252_skip"]
    .loc[start_date:]
    .drop(columns=["Natural Gas"])
)

returns_sub = (
    returns
    .loc[start_date:]
    .drop(columns=["Natural Gas"])
)

In [63]: def backtest_signal(signal,
                             returns_data,
                             frequency="monthly"):

    if frequency == "weekly":
        positions = weekly_positions(signal)

    elif frequency == "monthly":
        positions = monthly_positions(signal)

    else:
        raise ValueError(
            "frequency must be 'weekly' or 'monthly'"
        )

    positions = positions.shift(1)

    gross_exposure = positions.abs().sum(axis=1)

    positions = positions.div(
        gross_exposure,
        axis=0
    )

    positions = positions.fillna(0)

    strategy_returns = (
        positions * returns_data
    )

    portfolio_returns = (
        strategy_returns
        .sum(axis=1)
        .dropna()
    )

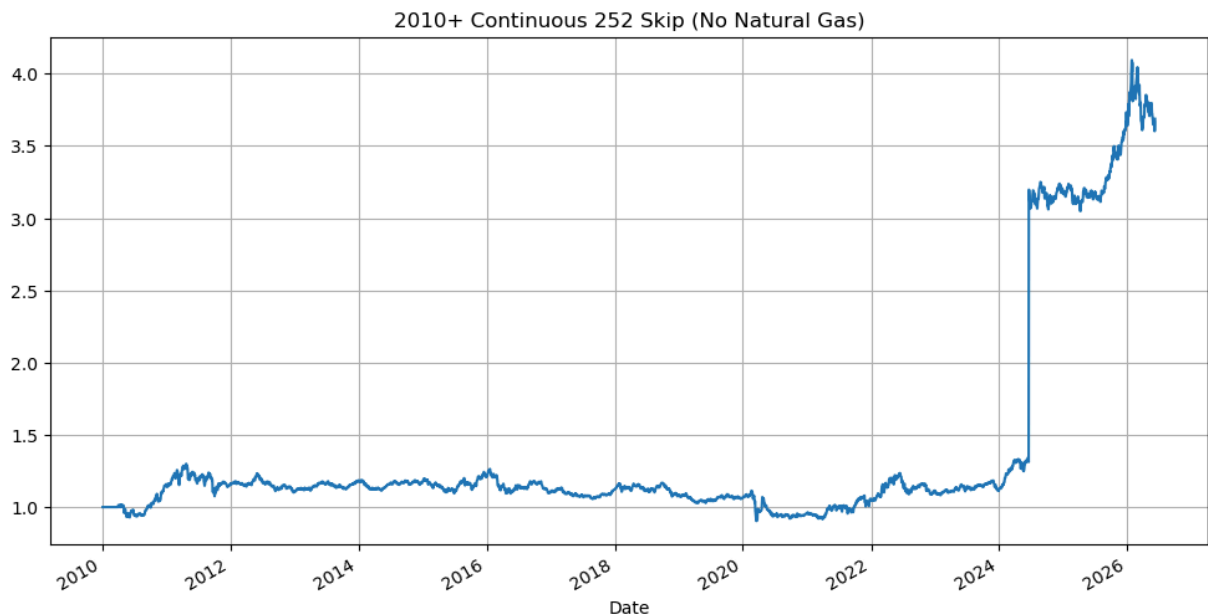
    return portfolio_returns
```

```
In [65]: ret_2010_no_ng = backtest_signal(
    signal,
    returns_sub,
    frequency="monthly"
)

performance_stats(ret_2010_no_ng)
```

```
Out[65]: Ann Return    0.082838
Ann Vol    0.368173
Sharpe    0.224997
Max DD    -0.304854
dtype: float64
```

```
In [67]: ((1 + ret_2010_no_ng).cumprod()).plot(
    figsize=(12,6),
    title="2010+ Continuous 252 Skip (No Natural Gas)"
)
plt.grid(True)
plt.show()
```



```
In [69]: energy = [
    "WTI Crude",
    "Brent",
    "Natural Gas"
]
```

```
In [71]: signal_no_energy = (
    signals["continuous"]["mom_252_skip"]
    .drop(columns=energy)
)

returns_no_energy = (
    returns
```

```
.drop(columns=energy)
)
```

```
In [73]: # =====
# 2010+ No Energy Backtest
# =====

start_date = "2010-01-01"

signal_no_energy = (
    signals["continuous"]["mom_252_skip"]
    .loc[start_date:]
    .drop(columns=energy)
)

returns_no_energy = (
    returns
    .loc[start_date:]
    .drop(columns=energy)
)

ret_2010_no_energy = backtest_signal(
    signal_no_energy,
    returns_no_energy,
    frequency="monthly"
)

performance_stats(ret_2010_no_energy)
```

```
Out[73]: Ann Return    0.091335
Ann Vol      0.381205
Sharpe      0.239595
Max DD      -0.255356
dtype: float64
```

```
In [75]: plt.figure(figsize=(12,6))

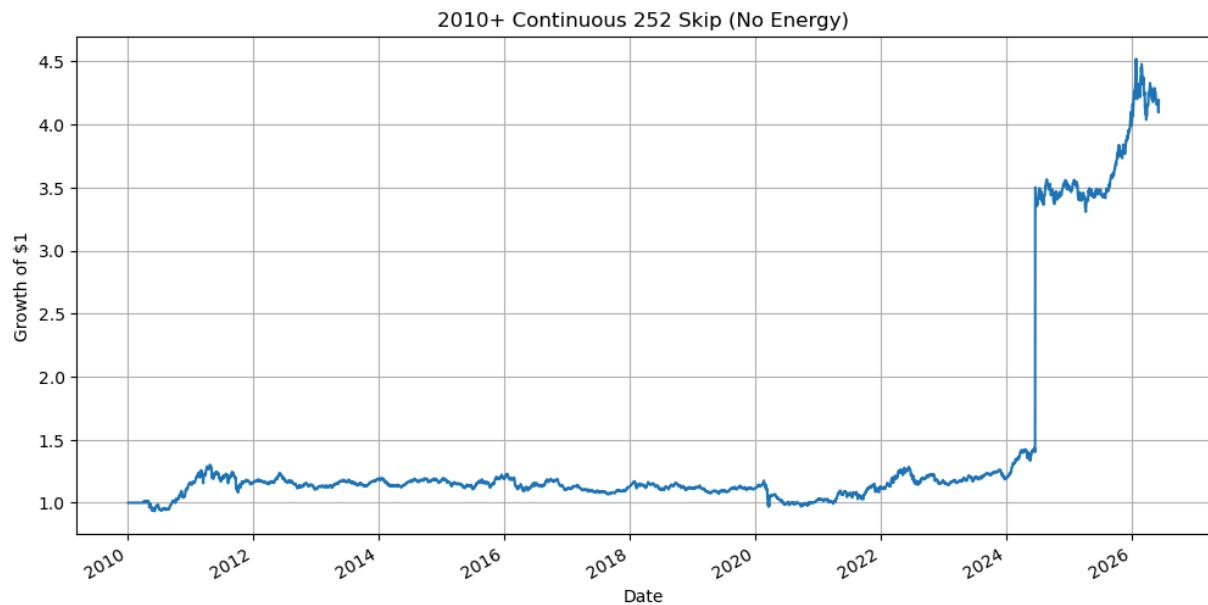
(
    1 + ret_2010_no_energy
).cumprod().plot()

plt.title(
    "2010+ Continuous 252 Skip (No Energy)"
)

plt.ylabel("Growth of $1")

plt.grid(True)

plt.show()
```

```
In [77]: plt.figure(figsize=(12,6))

(
    1 + continuous_monthly["mom_252_skip"]
).cumprod().plot(
    label="Full Sample"
)

(
    1 + ret_2010_no_ng
).cumprod().plot(
    label="2010+ No NG"
)

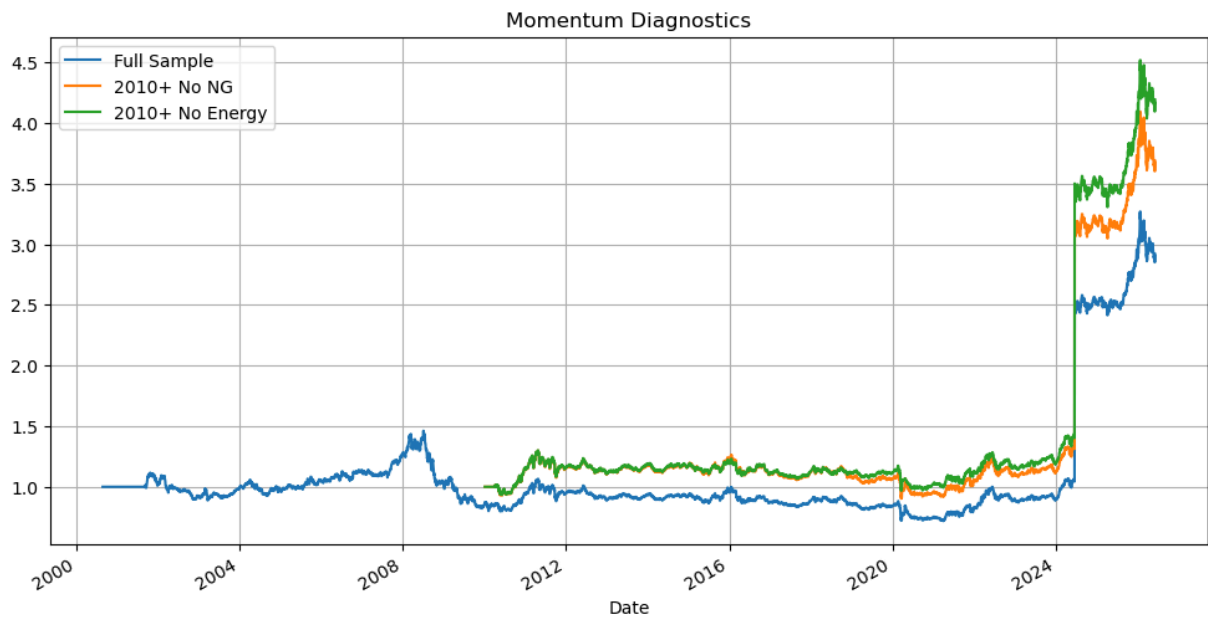
(
    1 + ret_2010_no_energy
).cumprod().plot(
    label="2010+ No Energy"
)

plt.legend()

plt.title(
    "Momentum Diagnostics"
)

plt.grid(True)

plt.show()
```



Weekly rebalance.

```
In [80]: ret_weekly = backtest_signal(
          signal_no_energy,
          returns_no_energy,
          frequency="weekly"
        )

          performance_stats(ret_weekly)
```

```
Out[80]: Ann Return    0.089842
          Ann Vol      0.349481
          Sharpe       0.257072
          Max DD       -0.238833
          dtype: float64
```

```
In [82]: plt.figure(figsize=(12,6))

          (
              1 + ret_2010_no_energy
          ).cumprod().plot(
              label="Monthly"
          )

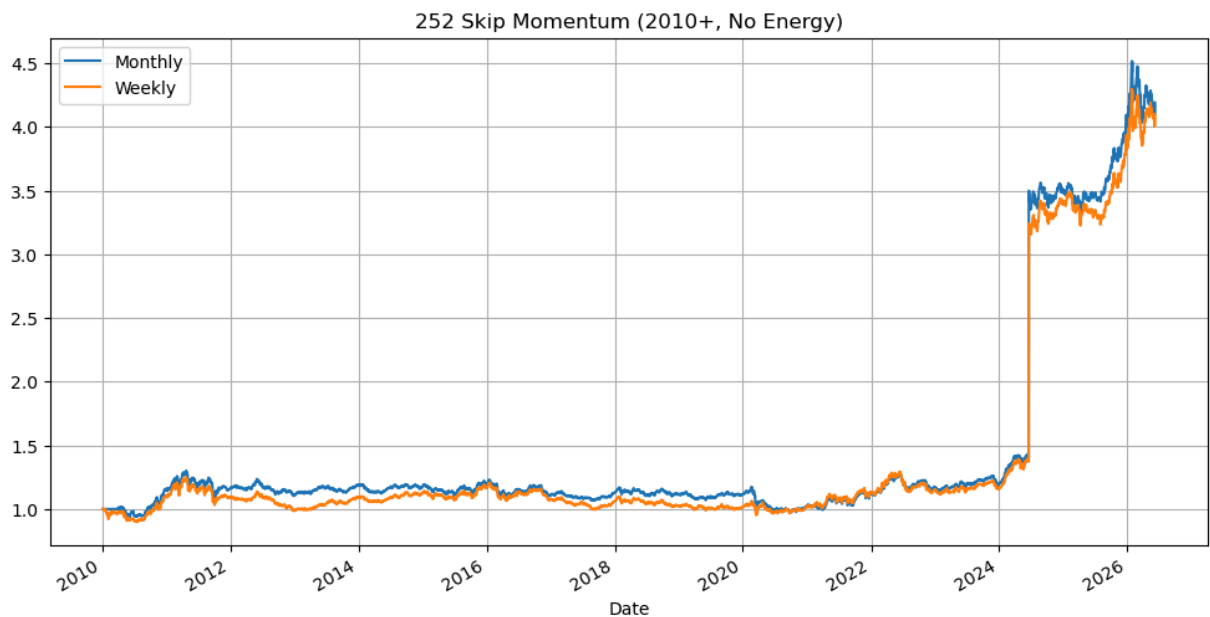
          (
              1 + ret_weekly
          ).cumprod().plot(
              label="Weekly"
          )

          plt.legend()

          plt.title(
              "252 Skip Momentum (2010+, No Energy)"
          )
```

```
plt.grid(True)

plt.show()
```



Let's test why energy doesn't work.

```
In [85]: energy_tests = {
    "full_energy": [],

    "brent_ng": [
        "WTI Crude"
    ],

    "brent_only": [
        "WTI Crude",
        "Natural Gas"
    ],

    "ng_only": [
        "WTI Crude",
        "Brent"
    ]
}
```

```
In [87]: results = {}

for test_name, remove_assets in energy_tests.items():

    signal_test = (
        signals["continuous"]["mom_252_skip"]
        .loc["2010-01-01":]
        .drop(columns=remove_assets)
    )

    returns_test = (
        returns
```

```

        .loc["2010-01-01":]
        .drop(columns=remove_assets)
    )

    ret = backtest_signal(
        signal_test,
        returns_test,
        frequency="weekly"
    )

    results[test_name] = performance_stats(ret)

```

In [89]: `pd.DataFrame(results).T`

Out[89]:

	Ann Return	Ann Vol	Sharpe	Max DD
full_energy	0.085771	0.338468	0.253409	-0.238383
brent_ng	0.083452	0.343056	0.243262	-0.250097
brent_only	0.087393	0.343547	0.254384	-0.238549
ng_only	0.085787	0.348992	0.245814	-0.241432

```

In [91]: results = {}

for test_name, remove_assets in energy_tests.items():

    signal_test = (
        signals["continuous"]["mom_252_skip"]
        .loc["2010-01-01":]
        .drop(columns=remove_assets)
    )

    returns_test = (
        returns
        .loc["2010-01-01":]
        .drop(columns=remove_assets)
    )

    ret = backtest_signal(
        signal_test,
        returns_test,
        frequency="monthly"
    )

    results[test_name] = performance_stats(ret)

```

In [93]: `pd.DataFrame(results).T`

Out[93]:

	Ann Return	Ann Vol	Sharpe	Max DD
full_energy	0.080253	0.365396	0.219634	-0.324287
brent_ng	0.082987	0.371303	0.223502	-0.317460
brent_only	0.085570	0.374115	0.228726	-0.290890
ng_only	0.088643	0.378295	0.234321	-0.283494

Now let's build a few signal comparison tables.

```
In [96]: # =====
# Signal Comparison Table
# 2010+
# Weekly
# Continuous Signals
# Brent + Natural Gas
# =====

start_date = "2010-01-01"

remove_assets = [
    "WTI Crude"
]

signal_results = {}

for signal_name, signal_df in signals["continuous"].items():

    signal_sub = (
        signal_df
        .loc[start_date:]
        .drop(columns=remove_assets)
    )

    returns_sub = (
        returns
        .loc[start_date:]
        .drop(columns=remove_assets)
    )

    ret = backtest_signal(
        signal_sub,
        returns_sub,
        frequency="weekly"
    )

    signal_results[signal_name] = (
        performance_stats(ret)
    )

signal_table = (
    pd.DataFrame(signal_results)
    .T
```

```

        .sort_values(
            "Sharpe",
            ascending=False
        )
    )

signal_table

```

C:\Users\pcarg\AppData\Local\Temp\ipykernel_32940\2178896061.py:8: RuntimeWarning: invalid value encountered in scalar power

```
(1 + returns).prod()
```

C:\Users\pcarg\AppData\Local\Temp\ipykernel_32940\2178896061.py:8: RuntimeWarning: invalid value encountered in scalar power

```
(1 + returns).prod()
```

Out[96]:

	Ann Return	Ann Vol	Sharpe	Max DD
mom_252_skip	0.083452	0.343056	0.243262	-0.250097
mom_252	0.031544	0.151859	0.207716	-0.403922
mom_126	0.030199	0.462343	0.065316	-0.533641
mom_126_skip	0.016182	0.268358	0.060301	-0.565185
mom_63	-0.001588	0.202942	-0.007823	-0.496734
mom_63_skip	-0.005533	0.199672	-0.027712	-0.563794
mom_21	NaN	0.606904	NaN	-1.820564
mom_21_skip	NaN	0.707226	NaN	-1.954573

In [98]:

```

# =====
# Brent Only Universe
# =====

start_date = "2010-01-01"

remove_assets = [
    "WTI Crude",
    "Natural Gas"
]

signal_results = {}

for signal_name, signal_df in signals["continuous"].items():

    signal_sub = (
        signal_df
        .loc[start_date:]
        .drop(columns=remove_assets)
    )

    returns_sub = (
        returns
        .loc[start_date:]

```

```

        .drop(columns=remove_assets)
    )

    ret = backtest_signal(
        signal_sub,
        returns_sub,
        frequency="weekly"
    )

    signal_results[signal_name] = (
        performance_stats(ret)
    )

signal_table = (
    pd.DataFrame(signal_results)
    .T
    .sort_values(
        "Sharpe",
        ascending=False
    )
)

signal_table

```

C:\Users\pcarg\AppData\Local\Temp\ipykernel_32940\2178896061.py:8: RuntimeWarning: invalid value encountered in scalar power

(1 + returns).prod()

C:\Users\pcarg\AppData\Local\Temp\ipykernel_32940\2178896061.py:8: RuntimeWarning: invalid value encountered in scalar power

(1 + returns).prod()

Out[98]:

	Ann Return	Ann Vol	Sharpe	Max DD
mom_252_skip	0.087393	0.343547	0.254384	-0.238549
mom_252	0.035566	0.152765	0.232813	-0.385986
mom_126	0.033360	0.468449	0.071214	-0.540231
mom_126_skip	0.018285	0.269605	0.067822	-0.573391
mom_63	0.006558	0.216136	0.030344	-0.482283
mom_63_skip	0.002915	0.213524	0.013651	-0.552817
mom_21	NaN	0.626948	NaN	-1.901654
mom_21_skip	NaN	0.747053	NaN	-2.068631

In [100...]

```

# =====
# Forecast Correlations
# =====

signal_corr = pd.DataFrame({
    "21_skip": signals["continuous"]["mom_21_skip"].stack(),
    "63_skip": signals["continuous"]["mom_63_skip"].stack(),
    "126_skip": signals["continuous"]["mom_126_skip"].stack(),

```

```

    "252_skip": signals["continuous"]["mom_252_skip"].stack(),
}).corr()

signal_corr

```

Out[100...

	21_skip	63_skip	126_skip	252_skip
21_skip	1.000000	0.481164	0.243538	0.032696
63_skip	0.481164	1.000000	0.638991	0.366081
126_skip	0.243538	0.638991	1.000000	0.633806
252_skip	0.032696	0.366081	0.633806	1.000000

Let's try some mixes.

```

In [103... combined_252_126 = (
    signals["continuous"]["mom_252_skip"]
    +
    signals["continuous"]["mom_126_skip"]
) / 2

```

```

In [105... combined_252_63 = (
    signals["continuous"]["mom_252_skip"]
    +
    signals["continuous"]["mom_63_skip"]
) / 2

```

```

In [107... combined_all = (
    signals["continuous"]["mom_21_skip"]
    +
    signals["continuous"]["mom_63_skip"]
    +
    signals["continuous"]["mom_126_skip"]
    +
    signals["continuous"]["mom_252_skip"]
) / 4

```

```

In [109... combined_signal = (
    combined_252_126
    .loc["2010-01-01":]
    .drop(columns=["WTI Crude", "Natural Gas"])
)

returns_sub = (
    returns
    .loc["2010-01-01":]
    .drop(columns=["WTI Crude", "Natural Gas"])
)

ret = backtest_signal(
    combined_signal,
    returns_sub,
    frequency="weekly"
)

```



```
)
performance_stats(ret)
```

```
Out[109... Ann Return    0.066534
Ann Vol      0.318512
Sharpe       0.208891
Max DD       -0.405523
dtype: float64
```

Diversification benefit is absent. Basically just diluting.

```
In [112... combined = (
    signals["continuous"]["mom_126_skip"]
    +
    signals["continuous"]["mom_252_skip"]
) / 2
```

```
In [114... combined = (
    signals["continuous"]["mom_63_skip"]
    +
    signals["continuous"]["mom_252_skip"]
) / 2
```

```
In [116... # =====
# Combined Forecast 1
# 126 Skip + 252 Skip
# =====

combined_126_252 = (
    signals["continuous"]["mom_126_skip"]
    +
    signals["continuous"]["mom_252_skip"]
) / 2

signal_sub = (
    combined_126_252
    .loc["2010-01-01":]
    .drop(columns=["WTI Crude", "Natural Gas"])
)

returns_sub = (
    returns
    .loc["2010-01-01":]
    .drop(columns=["WTI Crude", "Natural Gas"])
)

ret_126_252 = backtest_signal(
    signal_sub,
    returns_sub,
    frequency="weekly"
)

stats_126_252 = performance_stats(
    ret_126_252
```

```
)
stats_126_252
```

```
Out[116... Ann Return    0.066534
Ann Vol      0.318512
Sharpe      0.208891
Max DD      -0.405523
dtype: float64
```

```
In [118... # =====
# Combined Forecast 2
# 63 Skip + 252 Skip
# =====

combined_63_252 = (
    signals["continuous"]["mom_63_skip"]
    +
    signals["continuous"]["mom_252_skip"]
) / 2

signal_sub = (
    combined_63_252
    .loc["2010-01-01":]
    .drop(columns=["WTI Crude", "Natural Gas"])
)

returns_sub = (
    returns
    .loc["2010-01-01":]
    .drop(columns=["WTI Crude", "Natural Gas"])
)

ret_63_252 = backtest_signal(
    signal_sub,
    returns_sub,
    frequency="weekly"
)

stats_63_252 = performance_stats(
    ret_63_252
)

stats_63_252
```

```
Out[118... Ann Return    0.068324
Ann Vol      0.324248
Sharpe      0.210716
Max DD      -0.383346
dtype: float64
```

```
In [120... comparison = pd.DataFrame({
    "252_skip": performance_stats(
        ret_2010_no_energy
    ),
    "126_252": stats_126_252,
```

```
"63_252": stats_63_252
}).T
comparison
```

Out[120...

	Ann Return	Ann Vol	Sharpe	Max DD
252_skip	0.091335	0.381205	0.239595	-0.255356
126_252	0.066534	0.318512	0.208891	-0.405523
63_252	0.068324	0.324248	0.210716	-0.383346

Let's verify our universe is what we thought it would be.

```
In [ ]:
```